# Applying Reinforcement Learning to 2D Precision Platformer Games

First Author
國立成功大學
P76151466
楊乃諺

Second Author
國立成功大學
P76151123
王家騏

## Abstract

*Reinforcement learning (RL) has achieved remarkable success in game playing, yet mastering 2D precision platformers remains challenging due to stringent jump timing, momentum control, and severe reward sparsity. In this project, we investigate policy learning for autonomous precision platforming without handcrafted movement heuristics. We construct a 2D platformer testbed and train an agent using Proximal Policy Optimization (PPO). Operating on a low-dimensional numerical state representation, we integrate structured reward shaping and curriculum learning to mitigate credit assignment issues and guide exploration across increasingly narrow obstacles. The anticipated outcome is an agent capable of acquiring continuous velocity control and adaptive jumping policies. We will evaluate performance using level completion rates, trajectory progress, death counts, and learning efficiency, providing practical insights into solving physics-constrained dynamic navigation tasks.*

## 1. Introduction

### 1.1. Motivation

Reinforcement Learning provides a framework where an agent learns decision-making strategies through continuous trial and error using reward signals. Video games serve as an ideal testbed due to clearly defined states, actions, and quantifiable feedback. Deep reinforcement learning frameworks (e.g., DQN [1], Gym Retro [4]) have proven effective at learning policies directly from observations.

However, existing standard benchmarks often lack the challenge of high-precision dynamic control. In a 2D precision platformer, simply moving forward is insufficient; an agent must master exact jumping timing, air control, horizontal deceleration, and precise landings on narrow platforms. This study investigates: Can a model-free reinforcement learning agent acquire precise spatial navigation and landing strategies without manual heuristics or hard-coded rules?

### 1.2. Problem Definition

We formulate the platformer navigation task as a Markov Decision Process (MDP), defined by the tuple $\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$, where $\mathcal{S}$ is the state space, $\mathcal{A}$ is the action space, $\mathcal{P}$ represents state transition probabilities, $\mathcal{R}$ denotes the reward function, and $\gamma \in [0,1)$ is the discount factor. At each time step $t$, the agent observes state $s_t$, executes action $a_t$, and receives a scalar reward $r_t$. The learning objective is to optimize a policy $\pi(a \mid s)$ that maximizes the expected cumulative discounted return:

$$G_t = \sum_{k=0}^{\infty} \gamma^k \, r_{t+k}$$

In this environment, the agent must navigate from the starting point to the target goal while avoiding fatal falls. A critical challenge lies in long-term credit assignment and delayed action consequences—for instance, mid-air steering decisions during a jump dictate the landing trajectory several frames into the future.

### 1.3. Challenges of 2D Precision Platformers

Precision platformers present several distinct learning challenges: **(1) Precise movement**, where millimeter deviations determine platform landing versus failure; **(2) Temporal dependency**, where outcomes depend on multi-frame action sequences (e.g., accelerating before jumping); **(3) Sparse rewards**, as unguided exploration leads predominantly to falls; and **(4) Generalization**, requiring policies to adapt to unseen layouts rather than memorizing fixed routes.

### 1.4. Existing Approaches & Proposed Solution

Prior deep RL frameworks such as Deep Q-Networks (DQN) [1] and Proximal Policy Optimization (PPO) [2] have succeeded across discrete gaming environments; however, value-based methods such as DQN are primarily designed for discrete action selection and may become less convenient when fine-grained control or stable policy updates are required. To address these limitations, we propose employing PPO as our core actor-critic learner. By

pairing PPO's clipped surrogate objective with structured reward shaping, compact state representations, and progressive curriculum learning, our approach stably guides the agent through physics-constrained obstacles without relying on handcrafted heuristics.

## 2. Method

### 2.1. Environment

The platformer elements comprise a player character, platforms, gravity, horizontal movement, jump dynamics, and collision/failure checks. Progression difficulty is scaled by varying platform distances, widths, elevation gaps, and movement precision tolerances.

### 2.2. State Representation

Instead of directly using raw images, the agent receives numerical information such as:

$$s_t = [x_p, y_p, v_x, v_y, x_g, y_g, x_1, y_1, w_1, \dots]$$

where $(x_p, y_p)$ and $(v_x, v_y)$ denote agent position and velocity, $(x_g, y_g)$ is the goal, and $(x_i, y_i, w_i)$ specify platform coordinates and widths. This low-dimensional representation focuses learning on decision dynamics rather than visual perception.

### 2.3. Action Space

We define a discrete action space covering directional movement and jumping:
$\mathcal{A}$ = {None,Left,Right,Jump,Left+Jump,Right+Jump}.

As an experimental variation, we will also evaluate an automated-jump baseline to investigate whether explicit jump timing control benefits policy stability.

### 2.4. PPO Agent

We employ Proximal Policy Optimization (PPO) [2] as the core policy learner. At each step $t$, the agent observes $s_t$, samples $a_t \sim \pi_\theta(a_t \mid s_t)$, executes the action to receive $r_t$, and stores transitions into a rollout buffer. The policy is optimized using the clipped surrogate objective to constrain step size:

$$L^{CLIP}(\theta) = E_t[\min(r_t,(\theta)A_t clip(r_t,(\theta), 1-\epsilon_1+\epsilon)A_t)]$$

where $r_t(\theta) = \frac{\pi_\theta(a_t|s_t)}{\pi_{\theta_{old}}(a_t|s_t)}$, and $\hat{A}_t$ is the generalized advantage estimator (GAE). The clipping mechanism prevents destructively large policy updates, ensuring stable policy convergence in dynamic control.

### 2.5. Reward Function

Reward shaping is critical to mitigate reward sparsity and guide exploration. We formulate a composite reward:

$$R_t = w_p \Delta x + w_g R_{goal} - w_d R_{death} - w_s R_{stuck}$$

where $\Delta x$ measures progress toward the goal, $R_{goal}$ awards milestone completions (reaching platforms $+1$, checkpoints $+5$, goal $+20$), $R_{death}$ penalizes falls $(-5)$, $R_{stuck}$ discourages stalling $(-0.1)$, and $w_*$ are balancing weights. Careful tuning of these weights is crucial to prevent reward hacking—such as repeatedly collecting intermediate progress rewards without reaching the goal—ensuring stable and goal-oriented trajectories.

### 2.6. Curriculum Learning

To alleviate the initial exploration bottleneck, we implement curriculum learning by gradually scaling obstacle difficulty across three stages: (1) flat terrain with wide platforms, (2) variable platform heights with moderate gaps, and (3) narrow platforms requiring precise momentum and launch timing.

## 3. Expected Results

We expect the agent to progressively transition from stochastic exploration to mastering continuous velocity control, precise jump timing, and landing maneuvers without manual heuristics. We expect reward shaping to improve sample efficiency and reduce failure rates compared with a sparse-reward baseline. Ultimately, the trained policy will be evaluated on unseen procedural configurations to demonstrate generalizable platforming capabilities rather than route memorization.

## References


[1] V. Mnih, K. Kavukcuoglu, D. Silver, et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, pp. 529–533, 2015.

[2] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal Policy Optimization Algorithms," arXiv:1707.06347, 2017.

[3] A.Nichol, V. Pfau, C. Hesse, O. Klimov, and J. Schulman, "Gotta Learn Fast: A New Benchmark for Generalization in RL," arXiv:1804.03720, 2018.

[4] OpenAI, "Gym Retro," 2018.

[5] OpenAI, "Procgen Benchmark," 2019.